arXiv:2306.09458v1 [cs.NI] 15 Jun 2023

A flexible algorithm to offload DAG applications for edge computing

Gabriel F. C. de Queiroz[a,b,*], José F. de Rezende[b], Valmir C. Barbosa[b]

[a] *Coordenação do Curso de Engenharia de Telecomunicações, Centro Federal de Educação Tecnológica Celso Suckow da Fonseca, Rua General Canabarro, 485, Sala E-204, Rio de Janeiro, 20271-110, RJ, Brazil*
[b] *Programa de Engenharia de Sistemas e Computação, COPPE, Universidade Federal do Rio de Janeiro, Centro de Tecnologia, Sala H-319, Rio de Janeiro, 21941-972, RJ, Brazil*

Abstract

Multi-access Edge Computing (MEC) is an enabling technology to leverage new network applications, such as virtual/augmented reality, by providing faster task processing at the network edge. This is done by deploying servers closer to the end users to run the network applications. These applications are often intensive in terms of task processing, memory usage, and communication; thus mobile devices may take a long time or even not be able to run them efficiently. By transferring (offloading) the execution of these applications to the servers at the network edge, it is possible to achieve a lower completion time (makespan) and meet application requirements. However, offloading multiple entire applications to the edge server can overwhelm its hardware and communication channel, as well as underutilize the mobile devices' hardware. In this paper, network applications are modeled as Directed Acyclic Graphs (DAGs) and partitioned into tasks, and only part of these tasks are offloaded to the edge server. This is the DAG application partitioning and offloading problem, which is known to be NP-hard. To approximate its solution, this paper proposes the FlexDO algorithm. FlexDO combines a greedy phase with a permutation phase to find a set of offloading decisions, and then chooses the one that achieves the shortest makespan. FlexDO is compared with a proposal from the literature and two baseline decisions,

*Corresponding author.
Email address: gabriel.queiroz@cefet-rj.br (Gabriel F. C. de Queiroz)

considering realistic DAG applications extracted from the Alibaba Cluster Trace Program. Results show that FlexDO is consistently only 3.9% to 8.9% above the optimal makespan in all test scenarios, which include different levels of CPU availability, a multi-user case, and different communication channel transmission rates. FlexDO outperforms both baseline solutions by a wide margin, and is three times closer to the optimal makespan than its competitor.

Keywords: Multi-access Edge Computing, DAG applications, Offloading

1. Introduction

The rise of new network applications and services in recent years, such as mobile social networks (Hu et al., 2015; Huang et al., 2017), live video streaming (Aliyu et al., 2018; Yaqoob et al., 2020; Jedari et al., 2021), and the metaverse (Wang et al., 2023), for example, imposes increasingly demanding communication and computing requirements on user devices in order to provide a fully connected, immersive, and low-latency experience (Parvez et al., 2018). These stringent processing, memory, and bandwidth requirements are in general not met by the mobile devices that run those applications.

Even where user devices, such as smartphones and tablets, are capable of running such applications, this can come at great cost: having a time-consuming execution, slowing down the response time to user commands, heating up the device, and depleting the battery (Wu et al., 2018). One possible solution to this problem is to transfer (offload) the applications to a high-end server at the network core (Salaht et al., 2020), away from the end user. However, latency-sensitive applications can suffer from long transmission times if the server is located too far, resulting in a low quality of experience (Mao et al., 2017).

An additional problem with relying on central cloud servers is the ever-increasing computing and communication demands of applications. The more stringent their requirements and the number of mobile users running them, the greater the burden on the current network infrastructure and existing servers. Cisco (2020) estimated that nearly 300 million mobile applications will be downloaded by the end of 2023, including social media, business, and gaming. Also according to Cisco (2020), there will be over 13 billion mobile devices and connections by year end. The high number of mobile users demanding applications and data transmissions from/to cloud servers imposes

a real challenge in terms of bandwidth availability, transmission times, and end-to-end delay, which are critical for latency-sensitive applications (Salaht et al., 2020).

Multi-access Edge Computing (MEC) is thus key to enable the upcoming demand for new applications and services (Shi et al., 2016; Mao et al., 2017). MEC is a networking paradigm that brings part of the computational (memory, CPU) and transmission resources available in high-end servers at the network core, such as cloud data centers, to edge servers located at the network edge, usually connected to access points and close to the user (Shi et al., 2016). However, if there are many users fully depending on a single edge server, this may slow down the edge server and yield poor results in terms of reducing the completion time, also called makespan, of a latency-sensitive application.

To lighten the load on the edge server, it is possible to use it more efficiently, letting the mobile devices run parts of the applications. Since network applications are in general not monolithic and can be characterized as a set of multiple processing parts, called tasks, they are often described as Directed Acyclic Graphs (DAGs) (Jia et al., 2014) and task computation can be divided between mobile device and edge server. When a task runs on an edge server, it is said to have been offloaded to that server. This is a known research problem in MEC (Lin et al., 2020), called the DAG Application Partitioning and Offloading (DAPO) problem.

Many articles have already addressed this problem (Jia et al., 2014; Wang et al., 2016; Guo et al., 2019). However, there still remain three underexplored problems in the literature. First, CPU limitations on both mobile and edge devices, bandwidth sharing in data transmission, and task dependencies are often neglected (Salaht et al., 2020). Second, realistic data for DAG applications with more complex structures (Zhang et al., 2013) including more than a dozen tasks (Yang et al., 2020; An et al., 2022) are rarely found in the literature, and even the proposed solutions found in the literature fit only specific scenarios (Salaht et al., 2020). Finally, the so-called one-climb policy (Zhang et al., 2012; Yang et al., 2016; An et al., 2022) is always taken as being true. This policy says that the optimal offloading decision should never have non-offloaded tasks between offloaded tasks. In this paper, and to the best of our knowledge for the first time, this policy is shown to be not necessarily true.

This paper proposes the Flexible DAG Offloading (FlexDO) algorithm. FlexDO generates a reduced set of offloading decisions, each specifying which

device processes each task of a DAG application. The offloading decisions are generated by combining two steps. In the first, a greedy approach is followed to offload some tasks. To do this, FlexDO avoids transmitting large amounts of data by offloading tasks in pairs, since there is no transmission when two tasks run on the same device. In the second step, all possible offloading decisions of the remaining tasks are generated and the one resulting in the shortest makespan for the DAG application is selected.

The main contributions of this paper are as follows.

- The DAPO problem is formulated taking into account CPU and channel bandwidth sharing to calculate makespan.
- The one-climb policy is shown not to be a requirement for the optimal offloading decision on general DAGs, contrary to the common belief expressed in the literature. In fact, our results show that about 17% of optimal decisions for over a thousand DAG applications do not support this belief.
- The FlexDO heuristic is introduced to efficiently solve the problem by providing an offloading decision in reasonable time and reducing application makespan. Our results show that FlexDO achieves makespan values only 3.9% to 8.9% above the optimal in all test scenarios, which include single-user and multi-user cases, as well as multiple transmission data rates.
- Over a thousand realistic DAG applications are extracted from the Alibaba cluster trace and used to compare the performance of FlexDO with two baseline solutions and the main competitor from the literature. FlexDO outperforms all of them.

The remainder of this paper is organized as follows. Section 2 presents the state of the art of task offloading in MEC. In Section 3, the DAPO problem is defined. Section 4 discusses the proposed FlexDO heuristic solution. Section 5 shows how realistic DAG applications are obtained for experimentation and their composition. Our results are discussed in Section 6. Finally, Section 7 concludes this paper.

2. Related work

Efficiently distributing the execution of computational tasks across different devices is a common concern in many computing systems (Lee et al., 2013; Zhang et al., 2013; Yu, 2016; Zhang et al., 2019). Regarding edge computing, challenges arise within the DAPO problem, such as complex application structure, application partitioning (Lin et al., 2020), shortage of computing resources in mobile and edge devices, and highly dynamic transmission conditions (Shi et al., 2016). Many surveys have already reviewed general aspects of application offloading in the Internet of Things (IoT), as well as in multi-access edge and cloud computing (Dinh et al., 2013; Fernando et al., 2013; Khan et al., 2014; Yi et al., 2015; Mach and Becvar, 2017; Mao et al., 2017; Abbas et al., 2018; Parvez et al., 2018; Roman et al., 2018; Lin et al., 2020; Shakarami et al., 2020; Feng et al., 2022).

Kuang et al. (2019) use Lagrangian decomposition and convex optimization to jointly reduce energy consumption and execution delay in the offloading problem for MEC applications. Liao et al. (2021) propose a genetic algorithm for application offloading in a MEC-enabled ultra-dense cellular network, allowing applications to be offloaded and queued in a remote base station for sequential-only processing in order to reduce completion time and energy consumption. Wang et al. (2016) investigate optimization techniques in a computation offloading problem to minimize both completion time and energy consumption, exploring dynamic voltage and frequency scaling to lower the energy consumption of mobile devices. However, these works do not consider the partitioning of applications into smaller components (Kuang et al., 2019; Liao et al., 2021). Applications are offloaded in their entirety and this coarser granularity can burden the mobile device or edge server with full application executions. Even when partial offloading is considered, this is done by dividing the application into a number of bytes to be processed on the mobile device and the remainder to be processed on the edge server (as in Wang et al., 2016), but application partitioning cannot be done at any point (Mao et al., 2017), due to the structure of its internal components (tasks), state variables, and memory usage.

The following papers consider application partitioning into tasks and the data transmission between them in the offloading problem. An et al. (2022) address the offloading of sequential DAG applications and resource allocation in a MEC-assisted IoT system, reducing both the energy consumption of IoT devices and makespan under different channel conditions. Duan and Wu

(2021) propose an optimal partitioning decision for tree-based Deep Neural Networks (DNN) modeled as DAGs, to reduce DNN inference time of IoT applications. Jia et al. (2014) propose an offloading algorithm for sequential-only or parallel-only DAG applications, exploring data transmission load balancing in a mobile cloud scenario to reduce application makespan. These publications are among those that rely on overly simplified DAG structures: sequential and parallel tasks in Zhang et al. (2013); Jia et al. (2014); Duan and Wu (2021), DAG applications with only a few tasks (e.g., ten tasks in An et al., 2022), and purely synthetic DAGs related to no real application in Yang et al. (2020).

Guo et al. (2019) extend the prior work in Guo et al. (2018) and is the main related work regarding the scope of the present paper, as the authors consider real-world DAG applications, CPU shortage, and communication channel capacity in the DAPO problem. They formulate a Mixed Integer Linear Programming (MILP) problem to offload DAG applications in a MEC scenario. Then a congestion-aware heuristic is provided to solve this problem in reasonable time. Transmissions are scheduled on a First In, First Out (FIFO) basis and upcoming transmissions may be delayed in order to avoid parallelism in multi-user scenarios. A CPU assignment scheme is also provided, with task processing being allowed only if there are CPUs available.

Despite the contributions in Guo et al. (2019), CPU shortage is accounted for only on the edge server, thus assuming that mobile devices always have more CPUs than executable tasks at any moment. Another issue is that their algorithm gives up trying to offload when, at some point, the calculated makespan is greater than the makespan of the solution that runs entirely on the mobile device (without offloading). This can hinder the efficiency of their algorithm, because makespan reduction is not always monotonic.

This paper aims to solve the DAPO problem while considering the dependencies between tasks, the limitations of channel capacity for parallel transmissions, and the limitations of concurrent task processing with scarce CPUs. This is done in order to reduce the completion time (makespan) of real DAG applications taken from a data set with more complex structures than those found in the literature.

3. Problem definition

The MEC system model in this paper is described by Fig. 1, which depicts a single-user scenario. Applications run on end-user mobile devices,

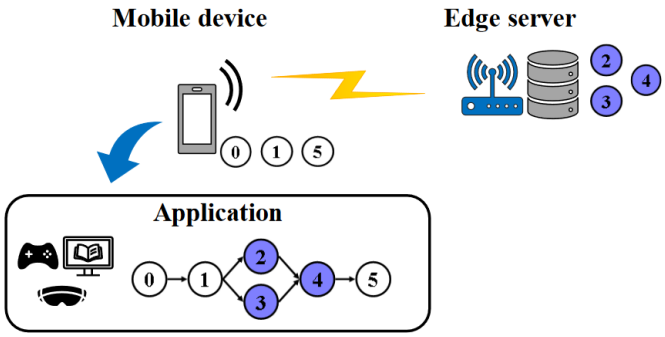


Fig. 1. MEC system model in the DAPO problem.

such as smartphones. These applications can be of any nature (Parvez et al., 2018), such as video streaming, augmented reality, virtual reality, e-health, e-learning, etc. Furthermore, applications can be partitioned into tasks, sequential or parallel, which are individual pieces of code to be executed on the mobile device or sent for remote processing on a nearby server at the network edge. Here it is assumed that both the mobile device and the edge server are capable of fully executing the applications.

Data transmission uses the wireless radio channel between the mobile device and the edge server. Still referring to Fig. 1, the application in the example has six tasks, and a possible offloading decision is shown. Tasks 2, 3 and 4 are offloaded to the edge server, while tasks 0, 1 and 5 remain assigned to the mobile device for local processing. Data transmissions are performed whenever task processing is offloaded and can occur from mobile device to edge server or in the opposite direction. The data transmitted correspond to the input parameters required to run a task, such as variables, which are passed from a preceding task to a succeeding task.

3.1. DAG application

Applications can be described as DAGs $G(V, E)$, where vertex set V is the set of tasks and directed-edge set E is the set of data transmissions (dependencies) between tasks. A task $i \in V$ can run on either the mobile device or the edge server. The number of parallel tasks that can be executed concomitantly, either on the mobile device or on the edge server, is not limited by the number of CPUs of each device. It is up to the scheduler of

each machine to allocate CPUs, as the tasks can share the same CPU, which means that a CPU does not necessarily run the same task 100% of the time. Also, with each task i a binary offloading decision o_i is associated, where $o_i = 0$ represents task execution on the mobile device and $o_i = 1$ represents execution on the edge server. The set $\{o_i, \forall i \in V\}$ is the offloading decision D for this application. Since a task is considered an indivisible part of an application, its processing cannot be interrupted and then migrated from one device to the other: a task must run entirely on the mobile device or on the edge server. In the same way, it is not possible to run a task on two or more CPUs at the same time to speed up processing.

An edge $(i, j) \in E$ indicates that the data passed from task i to task j are input parameters necessary for the execution of task j. A task can only start being executed if all data from all its incoming edges are received. In the case of multiple outgoing edges, as soon as a task i finishes its processing, it starts transmitting in parallel the corresponding data for each of those edges, requiring bandwidth sharing of the communication channel.

For the sake of simplicity, two assumptions are made. First, data transmissions cannot be interrupted and always finish successfully. Second, it is assumed that the underlying undirected graph is connected, and that there is only one task with no incoming edges (known as the initial task) and only one without outgoing edges (known as the ending task). These two tasks represent application start-up and result consumption. Both run necessarily on the end user's mobile device, as the application is started by the mobile device and the result of executing the application must be returned to it (Shu et al., 2017). Thus, neither the initial nor the ending task can be offloaded. Since the offloading decisions for these two tasks are already made in advance, the outgoing edges from the initial task and the incoming edges to the ending task are called anchors. The set of anchors is a subset of E.

Considering an application with N offloadable tasks, tasks t_0 and t_{N+1} are the initial and ending tasks, respectively. Application makespan is the total time elapsed from the beginning of t_0 to the end of t_{N+1}, when all tasks are done processing and there is no ongoing data transmission. Task processing times and data transmission times are defined next.

3.2. Task processing time

Task processing time is given by the time it takes a task to be fully executed on a single CPU. However, it may take longer if its allocated CPU is shared with other tasks. In fact, both the mobile device and the edge

server can host more tasks running in parallel than their number of CPUs. A new task starts processing immediately after receiving all data from all its incoming edges, regardless of the number of tasks already running. It is up to the task scheduler to run them evenly.

When a task is allocated to a single CPU without sharing it with other tasks, the processing time of task i is given by

$$t_i = \frac{n}{fc}, \tag{1}$$

where n is the task's number of operations, f is the clock speed (in cycles/second), and c is the CPU's computing capacity (in operations/cycle). However, by assuming CPU sharing, the processing time of a task increases as other tasks start running on the same CPU. If there are u CPUs on a device, up to u tasks can run in parallel without any sharing (100% CPU occupancy for each task). As soon as a new task starts, all running tasks receive only a fraction of a CPU, extending their remaining processing time by the factor $(u+1)/u$, as there are more tasks than CPUs available. Likewise, when a task finishes processing, each remaining task receives back a larger fraction of CPU occupancy, up to the maximum value of 100%, which implies no more CPU sharing. Here, this instantaneous fraction is called a CPU scale factor k_p, defined by

$$k_p = \max\left\{1, \frac{p}{u}\right\}, \tag{2}$$

where p is the number of tasks currently running in parallel. Since a task can never be sped up by using more than one CPU, k_p can never be less than 1, which means that the processing time given by Eq. (1) is also the task's minimal completion time.

Taking Eq. (2) into account, the remaining processing time for t_i, denoted by Δt_i, is updated whenever the CPU scale factor varies (that is, a new task starts/finishes running on that device) and the task receives a smaller/larger CPU share. The new value for the remaining processing time $\Delta t_i'$ is given by

$$\Delta t_i' = \frac{\Delta t_i k_p'}{k_p}, \tag{3}$$

where k_p represents the previous CPU scale factor, and k_p' is the new one. Having $k_p' > k_p$ means that a new task has begun running on that device; $k_p' < k_p$ means a task has ended.

It is worth noting that, as the number of tasks running on each device varies, the processing time for a task on the mobile device can become lower than that on the edge server due to their CPU scale factors, even if the edge server has a more powerful hardware. Thus, offloading tasks to the edge server whenever possible may result in a longer makespan.

3.3. Data transmission time

The wireless radio channel transmission rate (in bytes/second) is denoted by r and assumed to be the same for both uplink and downlink transmissions. Each edge (i, j) carries $d_{i,j}$ bytes from task i to task j. Thus, if there is no other ongoing transmission, the corresponding transmission time $T_{i,j}$ is given by

$$T_{i,j} = \frac{d_{i,j}}{r}. \tag{4}$$

However, by assuming a shared communication channel, multiple ongoing transmissions can take place. Channel bandwidth is shared evenly among the number of current data transmissions k_t, meaning all transmissions have the same priority. Whenever a new transmission starts or a previous transmission ends, the remaining time for all $d_{i,j}$ bytes to be transmitted, denoted by $\Delta T_{i,j}$, is updated. The new value for the remaining time $\Delta T'_{i,j}$ is given by

$$\Delta T'_{i,j} = \frac{\Delta T_{i,j} k'_t}{k_t}, \tag{5}$$

where $k'_t = k_t - 1$ if a transmission has ended or $k'_t = k_t + 1$ if a new one has started.

4. Heuristic approach

Providing an optimal solution to the DAPO problem in MEC is known to be difficult (Shu et al., 2017). Decisions based on integer linear optimization are present in the literature and they are proved to be NP-hard (Mao et al., 2017; Feng et al., 2022). All known exact approaches are therefore excessively time-consuming (Lin et al., 2020), so a heuristic is needed.

4.1. Overview

The following list summarizes the key observations underlying the design of FlexDO.

- Makespan reduction does not occur monotonically as more tasks are offloaded. Some offloading decisions, each one specifying where to run each task of a DAG application, can provide worse results than having no offloading at all. Even if offloading a task can shorten its processing time, this shortening can be offset by the time to transmit data from/to the task in question. For this reason, the proposed algorithm generates a set of offloading decisions, and tests each of them to select the one that results in the shortest makespan. As such, it does not rely on the last decision generated or on the decision that offloads the largest number of tasks.

- Some tasks are more relevant to makespan reduction than others, that is, the time difference (gain) between running a task locally or on the edge server varies from task to task in a DAG.

- Transmission times tend to weigh more heavily on makespan than processing times, but they can be canceled out. Task processing time only increases when there are more tasks than CPUs available (implying CPU sharing). However, as few as two transmissions in parallel are already enough for each one to have access to only half the available bandwidth (evenly shared), hence longer transmissions should probably be avoided by offloading the task pairs involved.

- The one-climb policy is not an absolute truth for the optimal offloading decision on a general DAG.

The one-climb policy is a recurring assumption in the literature (Zhang et al., 2012, 2013; Jia et al., 2014; Mao et al., 2017; Guo et al., 2019; Yan et al., 2020). This policy is based on the notion of a discontinuous offloading decision for a DAG application. If D is such a decision, then there exists at least one path in the DAG along which at least two tasks are marked for offloading by D while at least one other lying between them is not. That is, task processing goes down to the mobile device and up again to the edge server within that path. The one-climb policy states that, if D is a discontinuous offloading decision, then there will always be at least one continuous offloading decision D' resulting in a lower application makespan than D. Decision D', therefore, has at most one uplink transmission and one downlink transmission on every path in the DAG. Its proof for sequential DAG applications can be found in Yang et al. (2016).

Although the one-climb policy does hold for linear DAGs, in the general case it does not because of CPU limitations on the edge server and parallelism. It is possible for an edge server to be overloaded to the point where it is faster to process the task on the mobile device. Furthermore, when an offloading decision is forced to become continuous, it may require new data transmissions involving the tasks that were thus offloaded. These additional data transmissions can have an impact on the makespan.

One of the DAG applications from the data set used in this paper is shown in Fig. 2. Tasks in blue are the ones that should be offloaded to the edge server in the optimal offloading decision, which results in a makespan of 139.17 s. However, there is a discontinuity in a single DAG path, highlighted in red in the figure. When offloading task 17, the offloading decision becomes continuous, complying with the one-climb policy and canceling the transmission time of edges (11, 17) and (17, 18). However, this reduction negatively affects the total makespan of the DAG application, as it now performs the data transmissions on edges (6, 17), (7, 17), (8, 17), (9, 17), (12, 17), (13, 17), (14, 17), (15, 17), (17, 19), and (17, 21). The makespan of this continuous solution is 237.04 s, a deterioration of 70% in relation to the optimal decision. Thus, a discontinuous offloading scheme is not necessarily a bad decision. In fact, the results in Section 6 show that around 17% of the optimal decisions for our data violate the one-climb policy.

4.2. FlexDO

FlexDO generates a small set of offloading decisions to reduce the makespan of an application, beginning with the decision to offload no tasks and in the end testing each decision and choosing the one that provides the shortest makespan. This is done in two phases. In the first phase, it greedily searches for the edge with the longest data transmission time and offloads its two end tasks to obviate the data transmission between them. An important detail here is that if an edge shares an end task with an anchor, the transmission time with which it participates in this first phase is its original transmission time minus that of the anchors. This process continues on to the next edge with longest transmission time, now prioritizing edges with an end task already marked for offloading. If no edges remain connecting an offloaded task to a non-offloaded task, the edge with the longest transmission time is selected. As each new edge is selected in this sequence, it joins the previous ones in forming another offloading decision for later consideration.

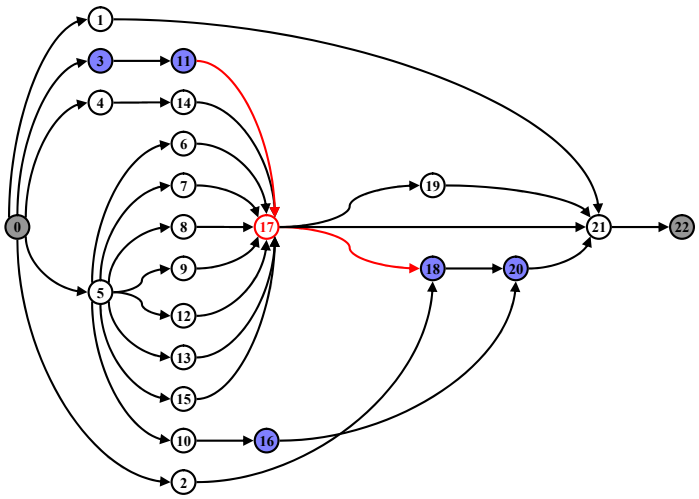
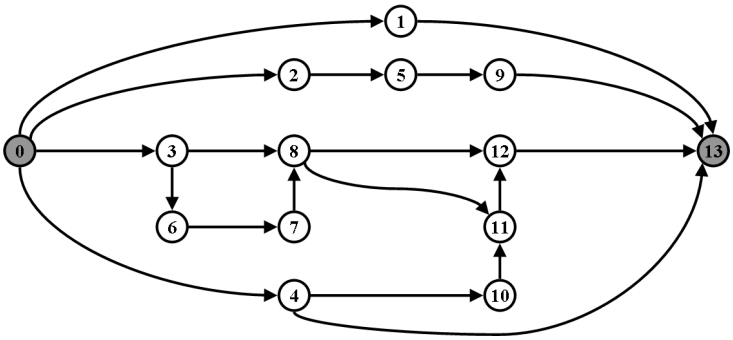


Fig. 2. Implications of the one-climb policy for general DAGs.

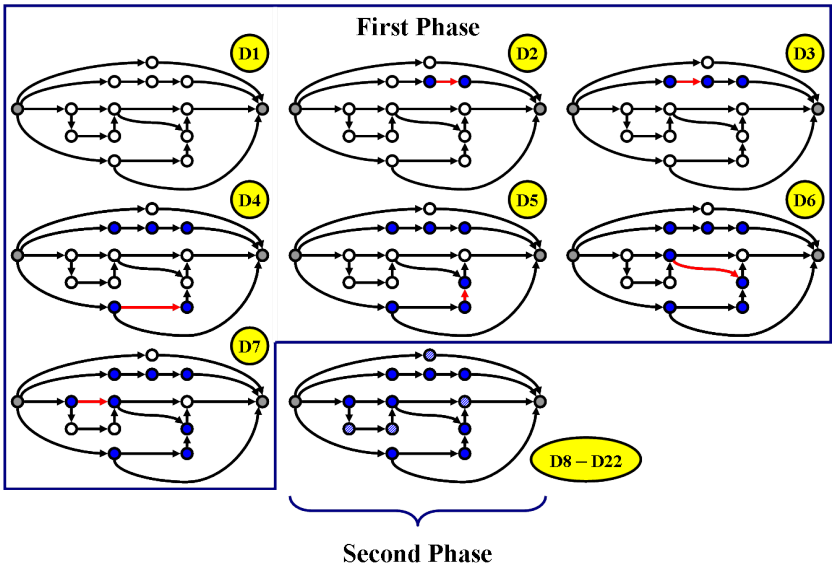
For a DAG application with N tasks, the first phase continues until s_I tasks have been marked for offloading, where $s_I < N$ is an input parameter. The second phase of FlexDO will then generate all $2^{s_{II}}$ possibilities of offloading decisions for the remaining $s_{II} = N - s_I$ tasks, while keeping the offloading marks from the first phase. After the second phase, FlexDO calculates the makespan associated with each decision and selects the one that results in the shortest makespan.

An example of how the FlexDO heuristic works is shown in Fig. 3. White tasks are marked to run on the mobile device and blue tasks to be offloaded to the edge server. Since the first and the last tasks always run on the mobile device and cannot be offloaded, they are grayed out. This example is for a DAG application with 12 offloadable tasks and $s_I = 8$; that is, the first phase lasts until 8 tasks have been marked to be offloaded.

The first offloading decision generated by FlexDO (D1) is the No Offloading decision. Now suppose edge (5, 9) has a transmission time of 80 s, the largest in the application. Since task 9 is anchored to task 13, the transmission time of edge (9, 13) must be subtracted from that of edge (5, 9). If data transmission on edge (9, 13) takes 12 s, then the time associated with



(a) A general DAG application.



(b) Offloading decisions generated in each phase.

Fig. 3. An example of how FlexDO operates.

edge (5, 9) for FlexDO operation is 68 s. Assuming this is still the most time-consuming transmission over all edges, tasks 5 and 9 are both marked for offloading (D2). Now the only available edge connected to tasks 5 or 9 is edge (2, 5), resulting in task 2 being marked for offloading (D3). From this point on, assume the first phase generates four more decisions (D4–D7), marking tasks 4, 10, 11, 8, and 3 for offloading.

At this point, FlexDO takes on its second phase and generates additional $2^4 = 16$ other offloading decisions in regard to tasks 1, 6, 7, and 12. First-phase D7 is one of them, so only 15 are actually generated. The makespan of each of D1–D22 is calculated, and the one yielding the shortest makespan is output by FlexDO.

Note that it is possible that there are not enough edges to go through in the first phase until the threshold s_I is reached. This happens when the DAG has many tasks associated only with anchor edges, as in the case of task 1 in the example. When this happens, an offloading gain is calculated for each task not yet marked for offloading, given by

$$G_k = t_k^m - t_k^e - \sum_{\substack{(i,j)\in E| \\ k\in\{i,j\}}} T_{i,j} \tag{6}$$

for task k, where t_k^m and t_k^e are the processing times of task k on the mobile device and on the edge server, respectively, and $T_{i,j}$ is the transmission time of edge (i, j), provided $k = i$ or $k = j$. Then, tasks are marked for offloading, from the highest gain to the lowest, which generates additional offloading decisions, until the threshold s_I is reached and the second phase begins. Pseudocode for FlexDO is given in Algorithm 1.

4.3. Computational complexity

The time complexity of Algorithm 1 can be broken up into four components.

1. The sorting of edges (by transmission times) and vertices (by gains), which is implicitly assumed. Considering that $|L|$ is $O(N^2)$, all the necessary sorting can be done in $O(N^2 \log N)$ time.
2. The loops in lines 6 and 14, taken together, iterate $s_I = O(N)$ times, each iteration requiring $O(1)$ time to complete.
3. The loop in line 18, which iterates 2^{N-s_I} times, spends $O(N)$ in each iteration. For $N - s_I = O(\log N)$, as in Section 6, this loop spends $O(N^2)$ time.

Algorithm 1: FlexDO.

```
1  let L be the set of non-anchor edges in E
2  for (i, j) ∈ L do
3  |  decrease T_{i,j} by every T_{u,v} such that (u, v) ∈ E \ L with u ∈ {i, j} or
   |    v ∈ {i, j}
4  let D be the offloading decision in which no task is marked for offloading
5  D ← {D}
6  while L ≠ ∅ and fewer than s_I tasks are marked for offloading in D do
7  |  if there exists (i, j) ∈ L such that ℓ is the only task in {i, j} not marked
   |    for offloading in D then
8  |  |  mark ℓ for offloading in D, provided T_{i,j} is longest over all such edges
9  |  else
10 |  |  mark both i and j for offloading in D, provided T_{i,j} is longest over L
11 |  D ← D ∪ {D}
12 |  L ← L \ {(i, j)}
13 let P be the set of tasks not marked for offloading in D
14 while fewer than s_I tasks are marked for offloading in D do
15 |  mark task k for offloading in D, provided G_k is greatest over P
16 |  D ← D ∪ {D}
17 |  P ← P \ {k}
18 for each of the 2^{s_II} possible joint markings for offloading of the s_II tasks not
     marked for offloading in D do
19 |  D' ← D
20 |  mark the s_II tasks for offloading in D' according to the joint marking being
   |    considered
21 |  D ← D ∪ {D'}
22 return the decision of shortest makespan in D
```

4. The makespan calculations in line 22, one for each of N offloading decisions, can be carried out for each decision as follows. Traverse the DAG breadth-first, in $O(N^2)$ time, and build a length-$O(N^2)$ event queue along the way. The events in this queue are the intertwined start and stop events of task executions and data transmissions. Sorting this queue by those events' time tags in ascending order in $O(N^2 \log N)$ time, and then processing it in $O(N^2)$ time, yields the DAG's makespan. Overall, $O(N^3 \log N)$ time is needed.

Adding up all four contributions yields an $O(N^3 \log N)$ time complexity, which is slightly higher than the $O(N^3)$ reported for HOA (Guo et al., 2019). Of course, for DAGs with $|L| = O(N \log N)$ this gets reduced to $O(N^2 \log^2 N)$, and for $|L| = O(N)$ it goes further down to $O(N^2 \log N)$.

5. Data analysis

FlexDO was developed to perform independently of DAG topology, based on realistic data. Most recent works deal with sequential (Ning et al., 2019; Wang et al., 2021), tree-based (Duan and Wu, 2021), or fan-in/fan-out DAGs (Mahmoodi et al., 2019; Mazouzi et al., 2019). Others do not even consider dependencies between the tasks of an application, targeting instead other aspects of the DAPO problem (Tran and Pompili, 2019; Peng et al., 2021), such as atomic tasks modeled as sequences of bits submitted in batch.

For the present study, the DAG applications were extracted from the Alibaba Cluster Trace Program from 2018.[1] This data set contains information from batch job tasks for millions of DAG applications submitted to a cluster of more than 4 000 machines.

5.1. Data set description

The data set describes its DAG applications in two files, one for the applications, which includes DAG topology, and another for instances, which have the internal specification of the tasks. These files contain information about memory size, percentage CPU utilization, start time, and end time of each instance, for example. As the instances are part of a task (i.e., a DAG vertex), and further details about their inner dependencies are omitted from the data set files, we summarize all instances for a given task as a single instance of processing time given by the average time of all instances.

Exact CPU and memory performance figures are also missing from the data set, but hardware information can be assumed based on Alibaba Cloud's instance family SCCG5, a general-purpose supercomputing cluster.[2] Table 1 shows the values used in this paper. Additional information about floating-point operations (FLOPs) per cycle is available in Dolbeau (2018).

Although there is information about how much memory is used by each task, it is not possible to infer from the data set how many bits are passed in data transmissions between tasks. Thus, here it is assumed that the total amount of data passed on all edges incoming to a given task is chosen randomly up to the total size of the data structure used by that task.

[1] https://github.com/alibaba/clusterdata

[2] https://www.alibabacloud.com/help/en/doc-detail/25378.htm#sccg5

Table 1: Hardware information for SCCG5 machines.

CPU architecture	Clock	No. of CPUs	RAM	FLOPs/cycle*
Intel Xeon Platinum 8163	2.5 GHz	48	384 GiB	32

* https://www.cpu-world.com/CPUs/Xeon/Intel-Xeon%208163.html

5.2. DAG preparation

The Alibaba data set contains more than 4 million DAG applications. We selected 1 322 of them from different parts of the data set, searching it while abiding by the following criteria.

- Include DAGs that have 18 to 28 tasks. The lower bound removes DAGs with overly simple topologies and the upper bound is applied so that it is still reasonable to find the optimal offloading decision through exhaustive enumeration.
- Remove DAGs with incomplete topology information.
- Remove DAGs without memory or CPU usage information.
- Keep only DAGs that have run successfully to completion.
- Interrupt the search after it has run for an hour.

Knowing that the data set includes DAG applications running on Alibaba clusters, it is reasonable to assume that these DAGs were submitted by users and that the results were returned to them. Thus, after obtaining the 1 322 DAGs, two tasks are added to each DAG to mimic the behavior of an application running on a mobile device. An initial task is connected to the tasks that do not have incoming edges, and an ending task is connected to those that do not have outgoing edges, as shown in Fig. 4. Neither task can be offloaded, hence they are both assigned zero processing time (any nonnegative constant would do here, as it affects the makespan of every DAG by the same amount). Their transmission times are generated as described in Section 5.1.

5.3. DAG properties

In addition to having realistic DAG applications, it is also important that DAGs be diverse in their structure, as the FlexDO heuristic is expected to

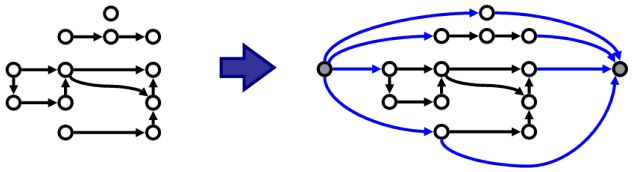


Fig. 4. Addition of the two extra tasks and their associated edges (in blue).

perform well in any scenario. Figure 5 shows how the DAG applications are distributed according to number of vertices, number of edges, and density. Density varies between 0, when a graph has no edges, and 1, when there is an edge for each of the $\frac{N(N-1)}{2}$ unordered pair of vertices.

As can be seen in the figure, the most common value of N is 23, which amounts to a little less than 25% of all DAGs. The remaining 75% are spread out less unevenly. As for the DAGs' numbers of edges and densities, these seem to thin out at the lower and higher values. This is reasonable, as these extremes generally bespeak little structured DAG: too few dependencies among tasks in the former case, too many in the latter.

6. Results

To evaluate the performance of FlexDO, three scenarios are used. The first scenario considers the variation in the number of CPUs available on the mobile device. The second considers this variation on the edge server. The third considers different transmission rates for the shared communication channel. In each of these scenarios, FlexDO is tested for three different values of s_{II}, namely, $s_{II} \in \{0, \log_2 N, \log_2 N^2\}$, following $s_{II} = N - s_I$. These values for s_{II} generate 1, N, or N^2 additional offloading decisions, respectively, in the second phase of Algorithm 1. Throughout these experiments, FlexDO is compared with the Heuristic Offloading Algorithm (HOA) from Guo et al. (2019), and with two baseline strategies (Full Offloading and No Offloading).

HOA always assumes an unlimited number of CPUs on the mobile device. If all CPUs on the edge server are busy, new tasks are queued until a CPU becomes available, rather than sharing CPUs with the tasks already running. Also, HOA does not perform more than one data transmission at a time, scheduling transmissions according to a FIFO scheme. These differences between the model underlying FlexDO and that underlying HOA imply that optimal decisions can be different for the same DAG. We follow Guo et al. (2019) faithfully as far as implementing and evaluating HOA are concerned.

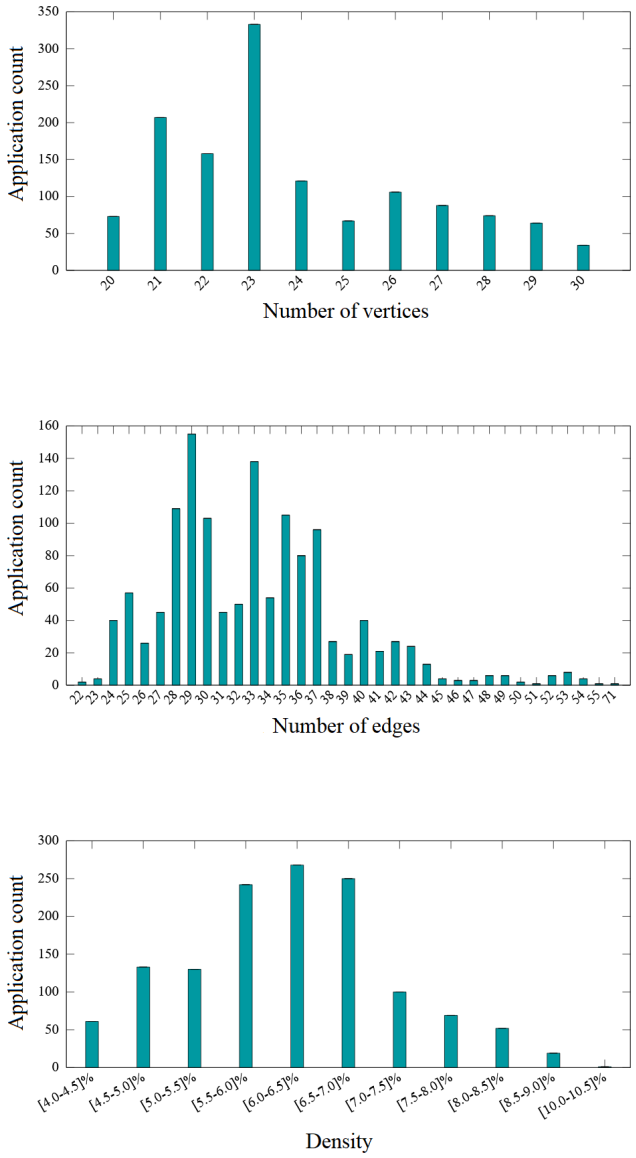


Fig. 5. Distributions of DAG properties.

Table 2: Hardware information for the mobile device and the edge server.

Type	CPU architecture	Clock	No. of CPUs	FLOPs/cycle*
Mobile device**	Qualcomm Snapdragon 865	1x2.84, 3x2.42, 4x1.8 GHz	8	8
Edge server***	Intel Xeon D-2100	2.0 GHz	16	32

* https://en.wikichip.org/wiki/flops#x86

** https://www.qualcomm.com/products/mobile/snapdragon/smartphones/snapdragon-8-series-mobile-platforms/snapdragon-865-5g-mobile-platform

*** https://www.intel.com/content/www/us/en/products/docs/processors/xeon/d-2100-brief.html

For the sake of fairness, results are therefore presented in terms of the relative difference between the heuristic makespan M_{heur} and the optimal makespan M_{opt}. FlexDO and the baseline strategies follow the modeling described in Section 3 and are compared with the corresponding optimal makespans. The HOA results are relative to the optimal makespans obtained within the system modeling of Guo et al. (2019).

Since the processing times of the Alibaba DAG applications refer to a large-scale computing cluster, these times are modified to better fit the hardware of commercial edge servers and current smartphones, whose characteristics are shown in the Table 2, according to

$$t_k^m = t_k^e \Big(\frac{f_e c_e}{f_m c_m} \Big) \tag{7}$$

and

$$t_k^e = t_k^c \Big(\frac{f_c c_c}{f_e c_e} \Big), \tag{8}$$

where t_k^m, f_m, and c_m are the processing time of task k in seconds, clock speed in Hz, and computing capacity in operations/cycle at the mobile device, respectively; t_k^e, f_e, and c_e are for the edge server; and t_k^c, f_c, and c_c are for the computing cluster.

6.1. Varying the number of CPUs of the mobile device

A mobile device may not have all its CPUs available for processing a DAG application, dedicating only part of them to it. This scenario assumes the availability of 2, 4, 8, or an unlimited number of CPUs on the mobile device, and the availability of all 16 CPUs of the edge server. The transmission rate of the wireless channel is assumed to be 20 Mbps, which is common in legacy Long Term Evolution (LTE) networks (Saliba et al., 2019).

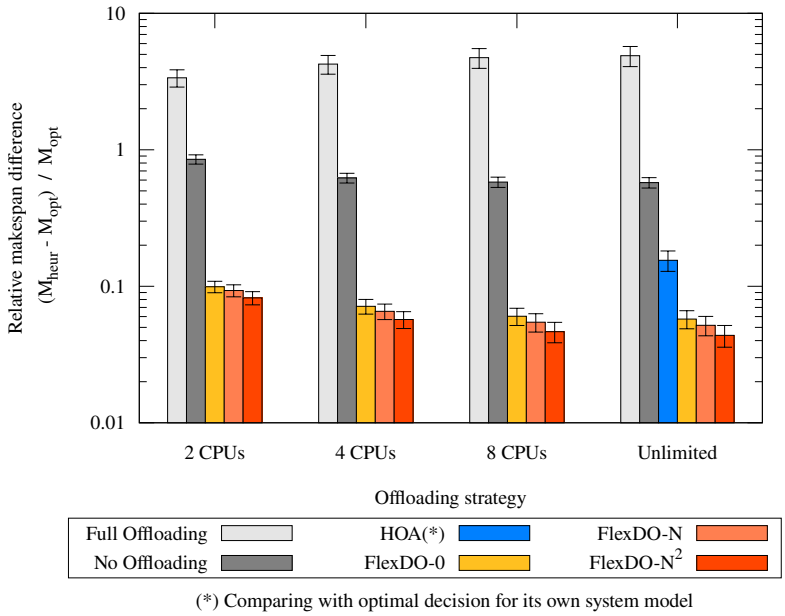


Fig. 6. Difference in relative makespan due to variation in the number of CPUs on the mobile device.

Figure 6 shows that FlexDO's performance remains stable regardless of the number of CPUs on the mobile device. FlexDO can only be meaningfully compared to HOA when both consider an unlimited number of CPUs at the mobile device, with FlexDO presenting results ranging from 4.3% to 5.7% above the optimal makespan, while HOA is 15.5% above its own optimal solution. It is worth noting that the fact that HOA always considers unlimited resources makes its makespan calculations optimistic, serving as an upper bound for the real makespan. And yet, even FlexDO-0, which tests a very limited set of offloading decisions, performs better. As shown in Fig. 6, the baseline solutions lead to unacceptably high makespans, since these strategies do not take into account task processing times or transmission times.

Still referring to Fig. 6, it must be noted that only in the Full Offloading case does the performance worsen as the number of CPUs increases on the mobile device. This is due to the fact that the increased number of CPUs does not benefit the Full Offloading decision, but rather only reduces the makespan of the optimal solution.

6.2. Varying the number of CPUs of the edge server (multi-user case)

This scenario investigates the impact on makespan of the number of CPUs available on the edge server. In a scenario with multiple mobile devices associated with a single edge server, a reduced number of CPUs at the edge may be available to process the DAG applications. This reduction is therefore a useful proxy to emulate the multi-user case.

Here, an unlimited number of CPUs on the mobile device is assumed for compatibility with HOA. On the edge server side, there are 1, 2, 4, or 16 CPUs (out of the 16 existing CPUs) available to a user. The transmission rate is kept 20 Mbps, as before.

Figure 7 shows the consistency of the three FlexDO versions, which are 5.5% to 6.8% above the optimal makespan when only one CPU is available on the edge server. These results become even better as more CPUs become available, achieving a makespan value only 4.3% to 5.7% above the optimal when all 16 CPUs are available. On the other hand, HOA's performance worsens relative to its optimal solution as more CPUs are available, going from 12.6% above the optimal makespan in the single-CPU case to 15.5% in the 16-CPU case. This contrast is due to the fact that FlexDO is more successful at choosing an offloading decision by testing a set of decisions, and insisting on the offloading process instead of going back to a baseline solution such as No Offloading. Also, both baseline solutions perform significantly worse than HOA or FlexDO.

In addition to the relative makespan results, it is worth delving deeper into the absolute makespan for this multi-user case. Table 3 shows the average makespan for all 1 322 DAG applications. Since these DAGs are quite diverse, ranging from quite simple to processing/transmission-intensive applications, a high deviation is expected. However, it is possible to see that, on average, FlexDO achieves better makespan values than HOA even with a smaller number of CPUs on the edge server. With 16 CPUs, HOA has an average makespan of 348.53 s, while FlexDO reaches 333.78 to 341.95 s with as few as two CPUs.

6.3. Varying the data transmission rate

Transmission rates may be reduced due to channel conditions (Mao et al., 2017), legacy equipment, and high demand in a crowded cell. On the other hand, higher transmission rates are expected in 5th Generation (5G) networks. So variation in data transmission has a potential to impact the offloading decision process. Here, an unlimited number of CPUs on the mobile

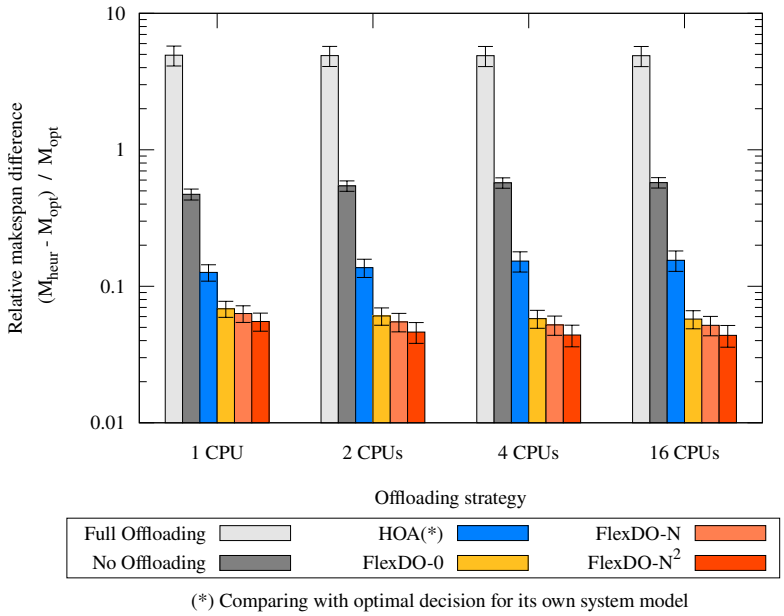


Fig. 7. Difference in relative makespan due to variation in the number of CPUs on the edge server.

device is considered while all 16 edge server CPUs are available to the DAG application. The transmission rates are 10 Mbps (degraded/crowded channel), 20 Mbps (legacy), and 100 Mbps (end-user data rate for 5G as in Parvez et al., 2018).

The transmission-avoidance aspect of FlexDO provides the most benefit from different transmission rates. Since FlexDO prioritizes offloading task pairs in order to nullify higher data transmissions between them, edges with fewer data to be transmitted are good candidates for migrating task processing from/to the edge server. Figure 8 shows how close FlexDO gets to the optimal solution. For a rate of 10 Mbps, FlexDO is 3.9% and 4.7% above the optimal makespan. With 100 Mbps, this performance remains stable, being only 5.2% to 8.9% above the optimal makespan. For HOA, on the other hand, the tendency is to move further and further away from the optimal makespan as the transmission rate increases, deviating by up to 24.4% from the optimal makespan with a transmission rate of 100 Mbps.

Table 3: Absolute makespan for HOA and FlexDO in the multi-user scenario.

	1 CPU	2 CPUs	4 CPUs	16 CPUs
HOA	385.95 ± 41.48	354.52 ± 36.49	349.05 ± 35.73	**348.53 ± 35.68**
FlexDO-0	370.60 ± 40.33	**341.95 ± 36.72**	334.12 ± 35.86	333.31 ± 35.78
FlexDO-N	366.27 ± 39.35	**337.67 ± 35.91**	329.77 ± 35.04	329.05 ± 34.96
FlexDO-N^2	361.53 ± 38.55	**333.78 ± 35.48**	326.37 ± 34.70	325.79 ± 34.65

Considering the evolution of future networks in terms of transmission rate, a greater separation from the optimal solution as the rate increases may lead to poor offloading decisions by HOA. This behavior is due to the fact that HOA seeks a monotonically decreasing makespan reduction, and then resorts to the No Offloading decision immediately when this monotonic reduction does not occur.

7. Concluding remarks

Given the results, it is clear that limiting FlexDO to neither the one-climb policy nor the monotonic reduction of makespan, and moreover generally not offloading tasks one by one, are beneficial in terms of obtaining average makespans lower than those of HOA in all scenarios. Furthermore, even though FlexDO does not calculate the impact of parallel transmissions or concurrent processing while generating decisions, these factors are duly considered when testing a small set of offloading decisions and selecting the decision that provides the shortest makespan. Also, FlexDO tries to offload those tasks that contribute most to reducing makespan, which has to do with the difference in processing time on the mobile device and on the edge server. It also cancels the highest data transmission times by keeping task pairs together on either the mobile device or the edge server. All these features refer back to the design principles behind FlexDO in Section 4, highlighting the importance of, as far as possible, considering all aspects that can possibly influence performance.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

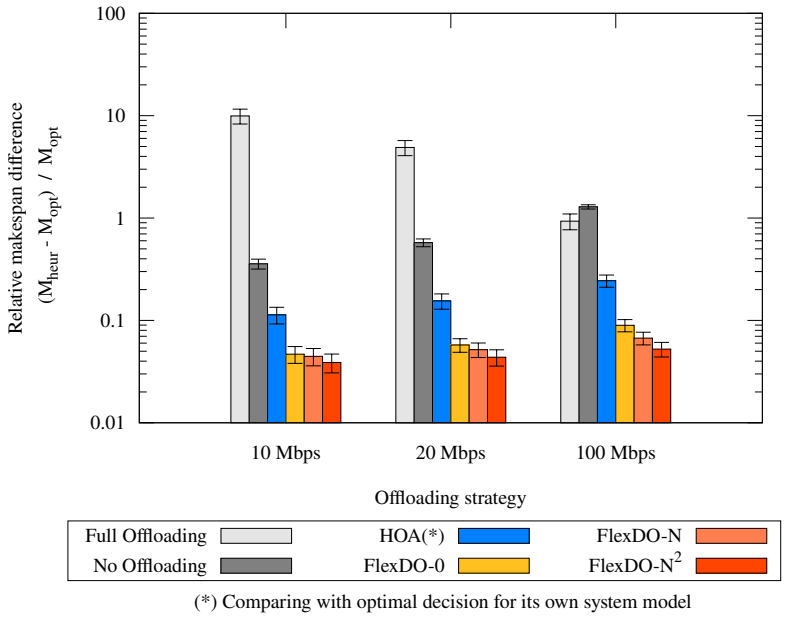


Fig. 8. Difference in relative makespan due to variation in data transmission rate.

Acknowledgments

We acknowledge partial support from Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES), and a BBP grant from Fundação Carlos Chagas Filho de Amparo à Pesquisa do Estado do Rio de Janeiro (FAPERJ). This work was also supported by MCTIC/CGI.br/São Paulo Research Foundation (FAPESP) through projects Slicing Future Internet Infrastructures (SFI2) – grant number 2018/23097-3, Smart 5G Core And MUltiRAn Integration (SAMURAI) – grant number 2020/05127-2 and Programmable Future Internet for Secure Software Architectures (PROFISSA) - grant number 2021/08211-7.

References

Abbas, N., Zhang, Y., Taherkordi, A., Skeie, T., 2018. Mobile edge computing: A survey. IEEE Internet of Things Journal 5, 450–465.

Aliyu, A., Abdullah, A.H., Kaiwartya, O., Cao, Y., Lloret, J., Aslam, N., Joda, U.M., 2018. Towards video streaming in IoT environments: Vehicular communication perspective. Computer Communications 118, 93–119.

An, X., Fan, R., Hu, H., Zhang, N., Atapattu, S., Tsiftsis, T.A., 2022. Joint task offloading and resource allocation for IoT edge computing with sequential task dependency. IEEE Internet of Things Journal 9, 16546–16561.

Cisco, 2020. Cisco annual Internet report (2018–2023) white paper. [Online]. https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html.

Dinh, H.T., Lee, C., Niyato, D., Wang, P., 2013. A survey of mobile cloud computing: Architecture, applications, and approaches. Wireless Communications and Mobile Computing 13, 1587–1611.

Dolbeau, R., 2018. Theoretical peak FLOPS per instruction set: A tutorial. The Journal of Supercomputing 74, 1341–1377.

Duan, Y., Wu, J., 2021. Computation offloading scheduling for deep neural network inference in mobile computing, in: 29th IEEE/ACM International Symposium on Quality of Service (IWQoS), pp. 1–10.

Feng, C., Han, P., Zhang, X., Yang, B., Liu, Y., Guo, L., 2022. Computation offloading in mobile edge computing networks: A survey. Journal of Network and Computer Applications 202, 103366.

Fernando, N., Loke, S.W., Rahayu, W., 2013. Mobile cloud computing: A survey. Future Generation Computer Systems 29, 84–106.

Guo, K., Yang, M., Zhang, Y., 2018. Computation offloading over a shared communication channel for mobile cloud computing, in: IEEE Wireless Communications and Networking Conference (WCNC), pp. 1–6.

Guo, K., Yang, M., Zhang, Y., Jia, X., 2019. Efficient resource assignment in mobile edge computing: A dynamic congestion-aware offloading approach. Journal of Network and Computer Applications 134, 40–51.

Hu, X., Chu, T.H.S., Leung, V.C.M., Ngai, E.C.H., Kruchten, P., Chan, H.C.B., 2015. A survey on mobile social networks: Applications, platforms, system architectures, and future research directions. IEEE Communications Surveys & Tutorials 17, 1557–1581.

Huang, Q., Ang, P., Knowles, P., Nykiel, T., Tverdokhlib, I., Yajurvedi, A., Dapolito, P., Yan, X., Bykov, M., Liang, C., Talwar, M., Mathur, A., Kulkarni, S., Burke, M., Lloyd, W., 2017. SVE: Distributed video processing at Facebook scale, in: 26th Symposium on Operating Systems Principles (SOSP), pp. 87–103.

Jedari, B., Premsankar, G., Illahi, G., Di Francesco, M., Mehrabi, A., Ylä-Jääski, A., 2021. Video caching, analytics, and delivery at the wireless edge: A survey and future directions. IEEE Communications Surveys & Tutorials 23, 431–471.

Jia, M., Cao, J., Yang, L., 2014. Heuristic offloading of concurrent tasks for computation-intensive applications in mobile cloud computing, in: 33rd IEEE International Conference on Computer Communications (INFOCOM) Workshop on Mobile Cloud Computing, pp. 352–357.

Khan, A.u.R., Othman, M., Madani, S.A., Khan, S.U., 2014. A survey of mobile cloud computing application models. IEEE Communications Surveys & Tutorials 16, 393–413.

Kuang, Z., Li, L., Gao, J., Zhao, L., Liu, A., 2019. Partial offloading scheduling and power allocation for mobile edge computing systems. IEEE Internet of Things Journal 6, 6774–6785.

Lee, K., Lee, J., Yi, Y., Rhee, I., Chong, S., 2013. Mobile data offloading: How much can WiFi deliver? IEEE/ACM Transactions on Networking 21, 536–550.

Liao, Z., Peng, J., Xiong, B., Huang, J., 2021. Adaptive offloading in mobile-edge computing for ultra-dense cellular networks based on genetic algorithm. Journal of Cloud Computing 10, 15.

Lin, H., Zeadally, S., Chen, Z., Labiod, H., Wang, L., 2020. A survey on computation offloading modeling for edge computing. Journal of Network and Computer Applications 169, 102781.

Mach, P., Becvar, Z., 2017. Mobile edge computing: A survey on architecture and computation offloading. IEEE Communications Surveys & Tutorials 19, 1628–1656.

Mahmoodi, S.E., Subbalakshmi, K., Uma, R.N., 2019. Spectrum-Aware Mobile Computing. Springer, Cham, Switzerland.

Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.B., 2017. A survey on mobile edge computing: The communication perspective. IEEE Communications Surveys & Tutorials 19, 2322–2358.

Mazouzi, H., Achir, N., Boussetta, K., 2019. Elastic offloading of multitasking applications to mobile edge computing, in: 22nd International ACM Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM), pp. 307–314.

Ning, Z., Dong, P., Kong, X., Xia, F., 2019. A cooperative partial computation offloading scheme for mobile edge computing enabled Internet of Things. IEEE Internet of Things Journal 6, 4804–4814.

Parvez, I., Rahmati, A., Guvenc, I., Sarwat, A.I., Dai, H., 2018. A survey on low latency towards 5G: RAN, core network and caching solutions. IEEE Communications Surveys & Tutorials 20, 3098–3130.

Peng, J., Qiu, H., Cai, J., Xu, W., Wang, J., 2021. D2D-assisted multi-user cooperative partial offloading, transmission scheduling and computation allocating for MEC. IEEE Transactions on Wireless Communications 20, 4858–4873.

Roman, R., Lopez, J., Mambo, M., 2018. Mobile edge computing, Fog et al.: A survey and analysis of security threats and challenges. Future Generation Computer Systems 78, 680–698.

Salaht, F.A., Desprez, F., Lebre, A., 2020. An overview of service placement problem in fog and edge computing. ACM Computing Surveys 53, 65.

Saliba, D., Imad, R., Houcke, S., El Hassan, B., 2019. WiFi dimensioning to offload LTE in 5G networks, in: 9th IEEE Annual Computing and Communication Workshop and Conference (CCWC), pp. 521–526.

Shakarami, A., Ghobaei-Arani, M., Shahidinejad, A., 2020. A survey on the computation offloading approaches in mobile edge computing: A machine learning-based perspective. Computer Networks 182, 107496.

Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L., 2016. Edge computing: Vision and challenges. IEEE Internet of Things Journal 3, 637–646.

Shu, G., Zheng, X., Xu, H., Li, J., 2017. Cloudlet-assisted heuristic offloading for mobile interactive applications, in: 5th IEEE International Conference on Mobile Cloud Computing, Services, and Engineering (MobileCloud), pp. 66–73.

Tran, T.X., Pompili, D., 2019. Joint task offloading and resource allocation for multi-server mobile-edge computing networks. IEEE Transactions on Vehicular Technology 68, 856–868.

Wang, Y., Sheng, M., Wang, X., Wang, L., Li, J., 2016. Mobile-edge computing: Partial computation offloading using dynamic voltage scaling. IEEE Transactions on Communications 64, 4268–4282.

Wang, Y., Su, Z., Zhang, N., Xing, R., Liu, D., Luan, T.H., Shen, X., 2023. A survey on metaverse: Fundamentals, security, and privacy. IEEE Communications Surveys & Tutorials 25, 319–352.

Wang, Z., Zhao, D., Ni, M., Li, L., Li, C., 2021. Collaborative mobile computation offloading to vehicle-based cloudlets. IEEE Transactions on Vehicular Technology 70, 768–781.

Wu, C., Yang, B., Zhu, W., Zhang, Y., 2018. Toward high mobile GPU performance through collaborative workload offloading. IEEE Transactions on Parallel and Distributed Systems 29, 435–449.

Yan, J., Bi, S., Zhang, Y.J., Tao, M., 2020. Optimal task offloading and resource allocation in mobile-edge computing with inter-user task dependency. IEEE Transactions on Wireless Communications 19, 235–250.

Yang, S., Bei, X., Zhang, Y., Ji, Y., 2016. Application offloading based on R-OSGi in mobile cloud computing, in: 4th IEEE International Conference on Mobile Cloud Computing, Services, and Engineering (MobileCloud), pp. 46–52.

Yang, T., Chai, R., Zhang, L., Chen, Q., 2020. Worst case latency optimization-based joint computation offloading and scheduling for interdependent subtasks, in: 12th International Conference on Wireless Communications and Signal Processing (WCSP), pp. 1010–1015.

Yaqoob, A., Bi, T., Muntean, G.M., 2020. A survey on adaptive 360° video streaming: Solutions, challenges and opportunities. IEEE Communications Surveys & Tutorials 22, 2801–2838.

Yi, S., Li, C., Li, Q., 2015. A survey of fog computing: Concepts, applications and issues, in: Workshop on Mobile Big Data (Mobidata), pp. 37–42.

Yu, Y., 2016. Mobile edge computing towards 5G: Vision, recent progress, and open challenges. China Communications 13, 89–99.

Zhang, T., Chiasserini, C.F., Giaccone, P., 2019. TAME: An efficient task allocation algorithm for integrated mobile gaming. IEEE Systems Journal 13, 1546–1557.

Zhang, W., Wen, Y., Wu, D.O., 2013. Energy-efficient scheduling policy for collaborative execution in mobile cloud computing, in: 32nd IEEE International Conference on Computer Communications (INFOCOM), pp. 190–194.

Zhang, Y., Liu, H., Jiao, L., Fu, X., 2012. To offload or not to offload: An efficient code partition algorithm for mobile cloud computing, in: 1st IEEE International Conference on Cloud Networking (CLOUDNET), pp. 80–86.